FORMATION 1,5 JOUR · NIVEAU DÉBUTANT

Apache Kafka, du capteur au flux

Comprendre, installer et programmer une plateforme de streaming d'événements

offset 18232	offset 18233	offset 18234	offset 18235	offset 18236	offset 18237	offset 18238
serre-27	serre-07	serre-26	serre-35	serre-35	serre-23	serre-34
temperature 22,1 °C	co2 748 ppm	co2 733 ppm	temperature 24,0 °C	humidite 74,1 %	co2 830 ppm	co2 615 ppm

Trois demi-journées, une seule histoire

PARTITION 0

Comprendre

Jour 1 · 9 h 00 – 12 h 30

- Le problème que Kafka résout
- Historique, de LinkedIn à Kafka 4
- Quoi, pourquoi, quand ne pas l'utiliser
- Tout le vocabulaire : topic, partition, offset, groupe…

PARTITION 1

Construire

Jour 1 · 14 h 00 – 17 h 30

- Un cluster Kafka 4 avec Docker Compose
- Chaque ligne de configuration expliquée
- Kafbat UI pour voir ce qui se passe
- Premiers messages en CLI puis en Python

PARTITION 2

Pratiquer

Jour 2 · 9 h 00 – 12 h 30

- Cas pratique en binôme : un pipeline d'alertes
- Montée en charge, rejeu, pannes
- Correction collective à la fin

Pauses : 10 h 30 et 15 h 45, 15 minutes. Déjeuner 12 h 30 – 14 h 00.

FIL ROUGE NOTRE CLIENT FICTIF

Verdalis, une coopérative maraîchère qui a trop de capteurs

Verdalis cultive tomates et salades dans 40 serres. Chaque serre embarque des capteurs de température, d'humidité et de CO_2.

- Le chef de culture veut un **tableau de bord** en temps réel.
- Le technicien veut une **alerte** quand une serre dépasse 32 °C.
- L'agronome veut **archiver** tout l'historique.
- La data scientist veut **entraîner un modèle** de prévision de récolte.

40
serres

1 000
capteurs au total

1 / s
une mesure par capteur

86 M
mesures par jour

LE FIL CONDUCTEUR

À chaque notion, on se demandera : **qu'est-ce que ça change pour Verdalis ?** Demain, vous construirez vous-mêmes une partie de leur plateforme.

À la fin de la formation, vous saurez…

EXPLIQUER

Ce qu'est Kafka et quand l'utiliser

Et surtout quand ne pas l'utiliser.

NOMMER

Chaque brique du vocabulaire

Broker, topic, partition, offset, réplique, ISR, groupe, commit, KRaft.

INSTALLER

Un cluster de 3 nœuds avec Docker Compose

En comprenant chaque variable, y compris les listeners.

PROGRAMMER

Un producteur et un consommateur Python

Avec les bons réglages de fiabilité, et le comportement face aux pannes.

Votre poste doit être prêt avant 14 h

- **Docker** avec Compose v2 (`docker compose version`)
- **6 Go de RAM libres** : 3 nœuds Kafka + l'interface
- Ports libres : `8080`, `29092`, `39092`, `49092`
- **Python 3.10+** et `pip install confluent-kafka`
- Un éditeur de code et un terminal

COPIER

```
# Télécharger les images à l'avance (le Wi-Fi de la salle vous remercie
docker pull apache/kafka:4.3.1
docker pull kafbat/kafka-ui:v1.5.0

# Préparer Python
python -m venv .venv
source .venv/bin/activate      # Windows : .venv\Scripts\activate
pip install confluent-kafka
```

ATTENTION

Sur un réseau d'entreprise, un proxy peut bloquer Docker Hub. Testez le `docker pull` la veille.

PARTITION 0 JOUR 1 · MATIN

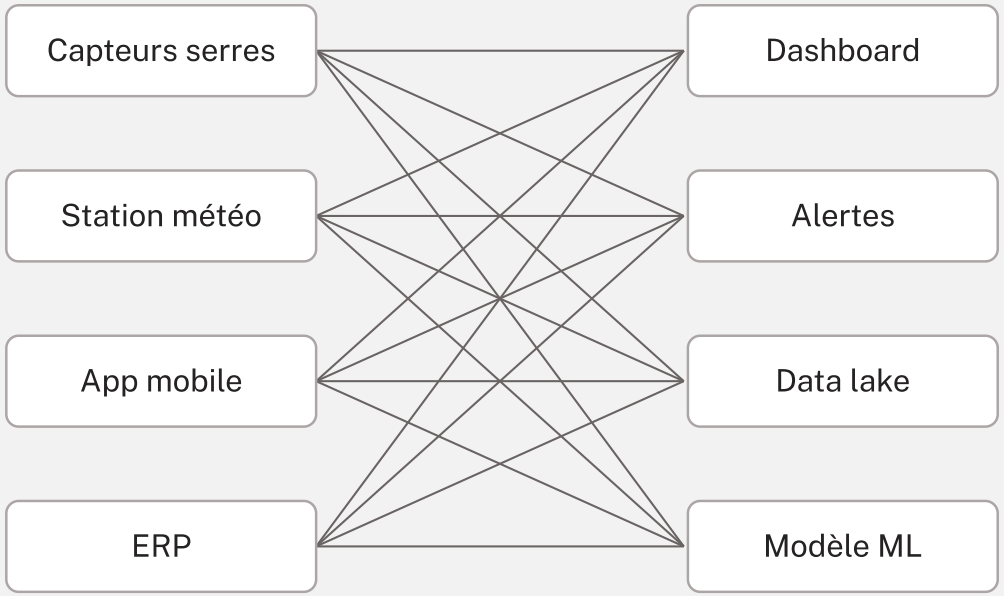
Comprendre

Le problème, l'histoire, les concepts. On ne tape pas une ligne de code ce matin, mais on pose tout ce qu'il faut pour cet après-midi.

P0 P1 P2

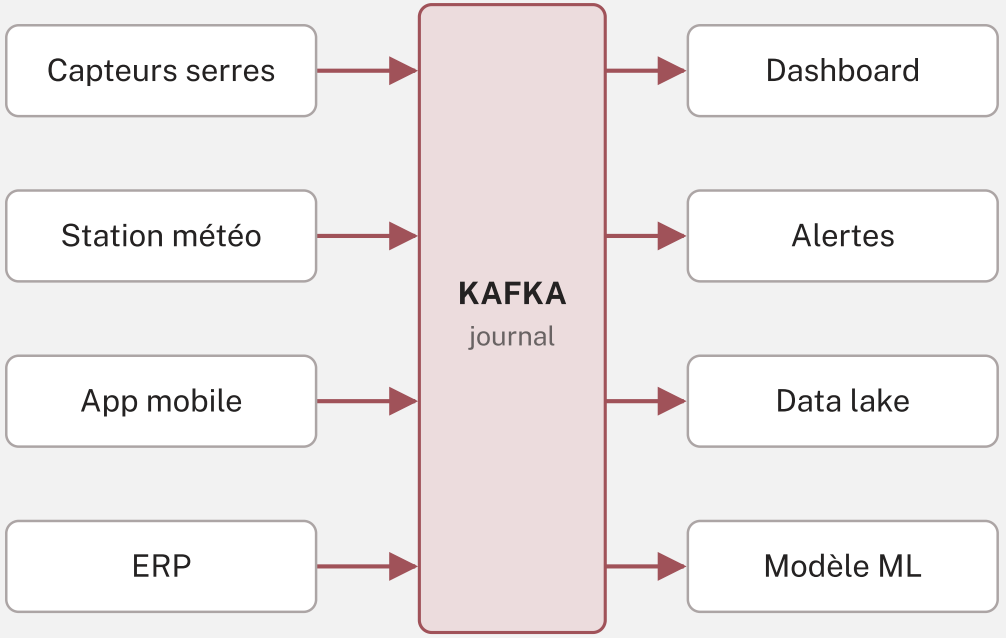
Chaque système parle directement à chaque système

AVANT · point à point



4 × 4 = 16 intégrations

APRÈS · un journal central



4 + 4 = 8 connexions

Quatre douleurs que Verdalis connaît déjà

COUPLAGE

Si le data lake tombe, les capteurs bloquent

L'émetteur attend la réponse du récepteur. Une panne se propage à toute la chaîne.

PICS DE CHARGE

Le lundi matin, tout arrive en même temps

Le récepteur le plus lent impose son rythme à tout le monde, ou perd des données.

PAS DE REJEU

Un bug dans l'alerte ? Les mesures d'hier sont perdues

Une fois transmise, la donnée n'existe plus nulle part. Impossible de rejouer.

EXPLOSION

N producteurs × M consommateurs

Chaque nouveau besoin ajoute des connexions, des formats et des scripts à maintenir.

Les producteurs écrivent dans un journal. Chacun le lit à son rythme.

- On **ajoute** toujours à la fin. On ne modifie jamais une ligne.
- Le journal **conserve** les événements pendant une durée choisie (7 jours par défaut).
- Chaque lecteur garde **son propre marque-page**. Lire ne supprime rien.
- Un lecteur en panne reprend là où il s'était arrêté.
- Un nouveau lecteur peut relire depuis le début.

DANS LA VRAIE VIE

Le **journal de bord d'un navire** : on y inscrit chaque événement à la suite, avec l'heure, sans jamais raturer. Le capitaine, le mécanicien et l'assureur le lisent chacun pour leurs besoins, au moment qui leur convient.

En anglais, ce journal s'appelle un **log** (commit log). Kafka est un log distribué.

HISTORIQUE 2010, LINKEDIN

Kafka est né du même problème que Verdalis, à une autre échelle

Vers 2010, LinkedIn doit faire circuler l'activité de ses membres (vues, clics, recherches) vers des dizaines de systèmes. Les outils de messagerie de l'époque ne tiennent pas la charge.

Trois ingénieurs, **Jay Kreps**, **Neha Narkhede** et **Jun Rao**, construisent un log distribué pensé pour le débit et la conservation.

POURQUOI « KAFKA » ?

Jay Kreps a expliqué avoir choisi le nom de l'écrivain Franz Kafka parce que c'est « un système optimisé pour l'écriture », et qu'il aimait son œuvre. Rien à voir avec le « kafkaïen ».

HISTORIQUE 15 ANS EN 6 DATES

De l'outil interne au standard du streaming

2011
Code publié en open source, entrée à l'incubateur Apache.

2012
Projet Apache de premier niveau.

2014
Les créateurs fondent Confluent.

2015–17
Kafka Connect, Kafka Streams, exactly-once, puis version 1.0.

2022
KRaft prêt pour la production (3.3) : Kafka se passe de ZooKeeper.

2025–26
Kafka 4.0 supprime ZooKeeper. 4.2 : files d'attente (share groups). 4.3.1 en juin 2026.

ATTENTION

Beaucoup de tutoriels en ligne utilisent encore ZooKeeper. Ils décrivent Kafka 3.x ou antérieur. Tout ce qu'on installe ici est en **KRaft**, le seul mode de Kafka 4.

Une infrastructure devenue banale dans les grandes entreprises

80 %+

des entreprises du Fortune 100 utilisent Kafka, selon le site du projet Apache

7 000 Md

de messages par jour chez LinkedIn en 2019

3 / an

versions majeures ou mineures visées par le projet, une tous les 4 mois

On le trouve dans la banque (transactions), la logistique (suivi de colis), l'industrie et l'agriculture (capteurs), le e-commerce (commandes) et partout où plusieurs systèmes doivent réagir aux mêmes événements.

QUOI DÉFINITION

Une plateforme distribuée de streaming d'événements

1 · PUBLIER ET S'ABONNER

Transporter

Des producteurs écrivent des événements, des consommateurs les lisent.

2 · STOCKER

Conserver

Les événements sont écrits sur disque, répliqués, et gardés aussi longtemps que voulu.

3 · TRAITER

Transformer

Filtrer, agréger, joindre des flux au fil de l'eau (Kafka Streams, etc.).

Distribuée : elle tourne sur plusieurs machines qui se partagent le travail et se remplacent en cas de panne.

Streaming : les données circulent en continu, au lieu d'être traitées par gros lots la nuit.

Un événement : « quelque chose s'est passé »

CHAMP	RÔLE	VERDALIS
Clé	Regroupe les événements liés	`serre-07`
Valeur	Le contenu (octets, souvent du JSON)	la mesure
Timestamp	Quand c'est arrivé	08:14:03
Headers	Métadonnées optionnelles	`source=lora`

DANS LA VRAIE VIE

Une **carte postale** : une adresse (la clé), un message (la valeur), un cachet de la poste (le timestamp).

```
// clé : "serre-07"
{
  "serre_id": "serre-07",
  "capteur_id": "serre-07-temperature",
  "type": "temperature",
  "valeur": 24.1,
  "unite": "°C",
  "ts": "2026-10-05T08:14:03Z"
}
```

QUOI COMPARAISON

Kafka n'est pas une file d'attente classique

	FILE CLASSIQUE (RABBITMQ, JMS…)	KAFKA
Après lecture	Message supprimé	Message conservé
Lecteurs	Un message va à un seul lecteur	Chaque groupe lit tout
Relire le passé	Impossible	Oui, pendant la rétention
Distribution	Le broker pousse (push)	Le consommateur tire (pull)
Ordre	Par file	Par partition
Point fort	Routage fin, tâches unitaires	Débit, rejeu, lecteurs multiples

DANS LA VRAIE VIE

La file, c'est la **boîte aux lettres** : on prend la lettre, elle n'y est plus. Kafka, c'est le **journal quotidien** : tout le monde lit le même numéro, et les archives restent consultables.

Depuis Kafka 4.2, les *share groups* ajoutent aussi un mode « file d'attente ». On reste ici sur le modèle classique des groupes.

Ce que Kafka apporte à Verdalis

BÉNÉFICE	CE QUE ÇA VEUT DIRE	POUR VERDALIS
Découplage	Producteurs et consommateurs ne se connaissent pas	Ajouter le modèle ML sans toucher aux capteurs
Débit	Écritures séquentielles sur disque, envoi par lots	86 M mesures par jour sans effort
Durabilité et rejeu	Les événements restent sur disque, répliqués	Corriger un bug d'alerte, puis rejouer la semaine
Scalabilité	On ajoute des partitions, des consommateurs, des machines	Passer de 40 à 400 serres
Tolérance aux pannes	Une machine tombe, une autre prend le relais	Pas de trou dans les données la nuit

Six familles d'usage

IOT

Télémétrie

Capteurs, véhicules, compteurs : beaucoup de petits messages en continu.

WEB

Activité utilisateur

Clics, recherches, paniers. L'usage d'origine chez LinkedIn.

INTÉGRATION

Capture de changements

Chaque modification d'une base est publiée comme événement (CDC).

OPS

Logs et métriques

Centraliser les journaux de milliers de serveurs avant indexation.

ARCHITECTURE

Microservices événementiels

« Commande passée » déclenche paiement, stock et e-mail sans appel direct.

DATA

Pipelines de données

Alimenter entrepôt, data lake et moteurs de recherche en temps réel.

ESPRIT CRITIQUE QUAND S'ABSTENIR

Kafka n'est pas la réponse à tout

BESOIN	PLUTÔT…
Question / réponse immédiate	Un appel HTTP ou gRPC
Chercher « toutes les mesures de la serre 7 en mars »	Une base de données
Quelques centaines de messages par jour	Une file simple, ou une table
Envoyer des vidéos ou gros fichiers	Stockage objet + un lien dans Kafka
Tâches avec priorité ou délai par message	Une file de tâches (RabbitMQ, SQS…)

ATTENTION

Kafka a un **coût d'exploitation** réel : supervision, mises à jour, sécurité, dimensionnement des disques. Pour une petite équipe, un service managé est souvent plus raisonnable.

Taille maximale d'un message par défaut : environ 1 Mo. On peut l'augmenter, mais c'est presque toujours un signal de mauvaise conception.

Le vocabulaire, dans l'ordre où la donnée voyage

1. Un **producteur** envoie un événement avec une **clé**…
2. …dans un **topic**, découpé en **partitions**, hébergé par des **brokers** qui forment un **cluster**.
3. L'événement reçoit un numéro, son **offset**, et il est **répliqué** sur d'autres brokers.
4. Des **consommateurs**, réunis en **groupes**, lisent les partitions et **commitent** leur progression.
5. Des **contrôleurs** (KRaft) coordonnent l'ensemble.

Les 20 slides suivantes détaillent chaque mot en gras, dans cet ordre.

CONCEPT BROKER · CLUSTER

Broker : un serveur Kafka. Cluster : plusieurs brokers ensemble.

- Un **broker** est un processus Kafka (une JVM) qui reçoit, stocke et sert les événements.
- Chaque broker a un identifiant unique : `node.id`.
- Un **cluster** regroupe plusieurs brokers qui se répartissent les données.
- Un client n'a besoin de connaître qu'**un** broker pour découvrir tous les autres. C'est le rôle des **bootstrap servers**.

DANS LA VRAIE VIE

Les **bureaux de poste d'une ville**. Chaque bureau stocke une partie du courrier. Si vous entrez dans n'importe lequel, on vous dit où se trouve votre colis.

CHEZ VERDALIS

3 brokers : `kafka-1`, `kafka-2`, `kafka-3`. C'est ce qu'on installe cet après-midi.

Topic : une catégorie nommée d'événements

- Un topic regroupe des événements de même nature : `verdalis.mesures`, `verdalis.alertes`.
- Les producteurs écrivent **dans** un topic, les consommateurs s'**abonnent** à un topic.
- Un topic peut avoir zéro, un ou cent consommateurs.
- Chaque topic a ses propres réglages : nombre de partitions, rétention, réplication.

DANS LA VRAIE VIE

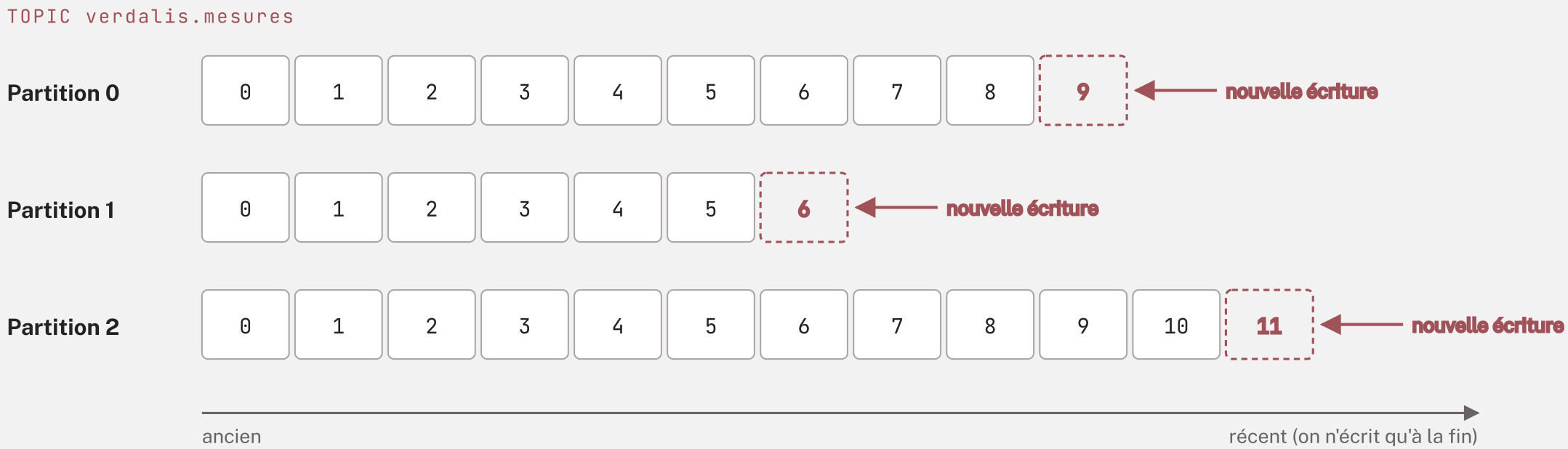
Un **rayon de bibliothèque** : « Romans », « Cuisine ». On range chaque livre dans le bon rayon, et chaque lecteur va au rayon qui l'intéresse.

BONNE PRATIQUE DE NOMMAGE

`domaine.entité` ou `domaine.entité.version`, en minuscules. Évitez de mélanger `.` et `_` : Kafka les confond dans ses métriques.

CONCEPT PARTITION

Partition : un morceau de topic, et un log à part entière



Un topic est découpé en **partitions**. Chacune est un log indépendant, stocké sur un broker. C'est l'unité de **parallélisme** de Kafka.

DANS LA VRAIE VIE

Les **caisses d'un supermarché** : plus il y a de caisses ouvertes, plus on sert de clients en même temps. Dans chaque file, l'ordre d'arrivée est respecté.

CONCEPT OFFSET

Offset : la position d'un événement dans sa partition

- Un entier qui commence à 0 et augmente de 1 à chaque écriture.
- Unique **dans une partition** : l'offset 5 existe dans P0, P1 et P2.
- Adresse complète d'un événement : `topic + partition + offset`.
- Un offset n'est jamais réutilisé, même quand les anciens messages sont supprimés.

DANS LA VRAIE VIE

Le **numéro de page** d'un livre. Pour reprendre votre lecture, vous notez la page, pas le contenu.

REGARDEZ EN BAS À DROITE

Cette présentation est un log. Chaque partie est une partition (`p0` , `p1` , `p2`), chaque slide a un offset qui repart à 0 dans chaque partie.

La clé décide de la partition

```
partition = hash(clé) % nombre_de_partitions

hash("serre-07") % 6  →  2   (toujours)
hash("serre-12") % 6  →  5   (toujours)
hash("serre-31") % 6  →  2   (même partition que serre-07)
```

- Même clé ⇒ même partition ⇒ **ordre garanti** pour cette clé.
- Sans clé : les messages sont répartis par lots entre les partitions (sticky partitioner).

DANS LA VRAIE VIE

Le **vestiaire d'un théâtre** : la première lettre de votre nom décide du guichet. Vous retrouvez toujours votre manteau au même endroit.

ATTENTION

Ajouter des partitions à un topic existant change le résultat du modulo. Les nouveaux messages de `serre-07` peuvent partir ailleurs. Choisissez le nombre de partitions dès la création.

CONCEPT ORDRE

L'ordre est garanti dans une partition, pas dans un topic

Le capteur de la serre 7 envoie 18 °C, puis 25 °C, puis 33 °C.

- Avec la clé `serre-07` : les trois mesures vont dans la même partition. Tout consommateur les lit dans cet ordre.
- Sans clé : elles peuvent tomber dans trois partitions. Le consommateur peut voir 33 °C avant 18 °C.

QUESTION À SE POSER

« Pour quel regroupement l'ordre compte-t-il ? » La réponse donne la clé. Chez Verdalis : par serre. Dans une banque : par compte. En e-commerce : par commande.

ATTENTION

Une clé trop concentrée (90 % des messages sur une seule clé) crée une partition « chaude » et annule le parallélisme.

CONCEPT RÉTENTION · COMPACTION

Combien de temps Kafka garde-t-il les données ?

Chaque partition est stockée en fichiers de **segments** (1 Go par défaut). Kafka supprime les segments anciens entiers, jamais un message isolé.

RÉGLAGE DU TOPIC	DÉFAUT	EFFET
`retention.ms`	7 jours	Âge maximal
`retention.bytes`	illimité	Taille max par partition
`cleanup.policy`	`delete`	ou `compact`

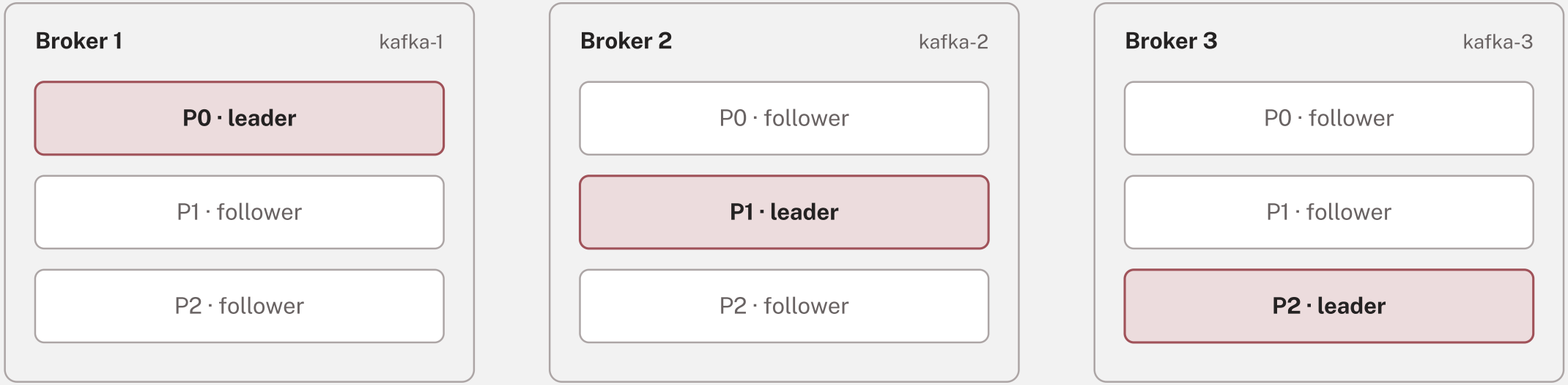
DANS LA VRAIE VIE

`delete` , c'est la **vidéosurveillance** qui efface les enregistrements de plus de 7 jours.
`compact` , c'est un **carnet d'adresses** : on ne garde que la dernière adresse connue de chaque personne (la dernière valeur par clé).

CHEZ VERDALIS

`verdalis.mesures` : 7 jours, suppression. Un topic `verdalis.serres.config` (seuils par serre) serait compacté.

Chaque partition existe en plusieurs copies



Les producteurs et les consommateurs parlent au leader. Les followers recopient.

Le **facteur de réplication** (RF) donne le nombre de copies. Avec RF = 3, on peut perdre 2 brokers sans perdre de données. Une copie est **leader**, les autres sont **followers**.

DANS LA VRAIE VIE

Un **acte notarié** : l'original chez le notaire (le leader), des copies certifiées dans deux autres études (les followers). Si l'étude brûle, une copie devient l'original.

Les répliques « à jour » : l'ISR

- **ISR** (In-Sync Replicas) : l'ensemble des copies qui ont bien suivi le leader. Le leader en fait partie.
- Un follower trop en retard (30 s par défaut) ou arrêté **sort** de l'ISR. Il y revient une fois rattrapé.
- Si le leader tombe, le nouveau leader est choisi **dans l'ISR**. Aucune donnée confirmée n'est perdue.
- `min.insync.replicas` : nombre minimal de copies à jour pour accepter une écriture sûre.

RF = 3, MIN ISR = 2	ÉCRITURE SÛRE ?
3 brokers en ligne	Oui
1 broker en panne	Oui
2 brokers en panne	Non : Kafka refuse

LA RÈGLE À RETENIR

RF = 3 et `min.insync.replicas` = 2. C'est la configuration de production la plus courante, et celle de notre cluster.

CONCEPT PRODUCTEUR

Le producteur : il envoie, mais par lots

- Il choisit la partition (via la clé), puis envoie au **leader** de cette partition.
- `produce()` ne part pas tout de suite : le message va dans un tampon.
- Le tampon part quand il est plein (`batch.size`) ou après un délai (`linger.ms`).
- Les lots peuvent être **compressés** (lz4, zstd…).
- En cas d'erreur temporaire, il **réessaie** tout seul.

DANS LA VRAIE VIE

Le **facteur** ne fait pas un aller-retour par lettre : il remplit sa sacoche, puis part en tournée. Attendre 20 ms de plus permet de livrer 100 fois plus.

ATTENTION

Si le programme se termine sans `flush()`, les messages encore dans le tampon sont perdus. C'est l'erreur n° 1 des débutants.

`acks` : quand considère-t-on un message comme écrit ?

ACKS	LE PRODUCTEUR ATTEND…	LATENCE	RISQUE DE PERTE
`0`	Rien. Il envoie et oublie.	Minimale	Élevé
`1`	Le leader a écrit.	Faible	Si le leader meurt avant réplication
`all`	Toutes les copies de l'ISR ont écrit.	Un peu plus	Quasi nul (avec min ISR = 2)

Idempotence (`enable.idempotence=true`) : si le producteur réessaie un envoi déjà reçu, le broker le reconnaît et ne l'écrit pas deux fois.

DANS LA VRAIE VIE

`acks=all` , c'est le **recommandé avec accusé de réception**. L'idempotence, c'est le numéro de suivi : La Poste sait que le deuxième envoi est un doublon.

CONCEPT CONSOMMATEUR

Le consommateur : il tire les messages, à son rythme

- Il **demande** les messages (pull) dans une boucle `poll()`.
- Il lit une partition dans l'ordre, offset après offset.
- Il note où il en est : le **commit d'offset**, stocké dans un topic interne, `__consumer_offsets`.
- Au redémarrage, il reprend au dernier offset commité.

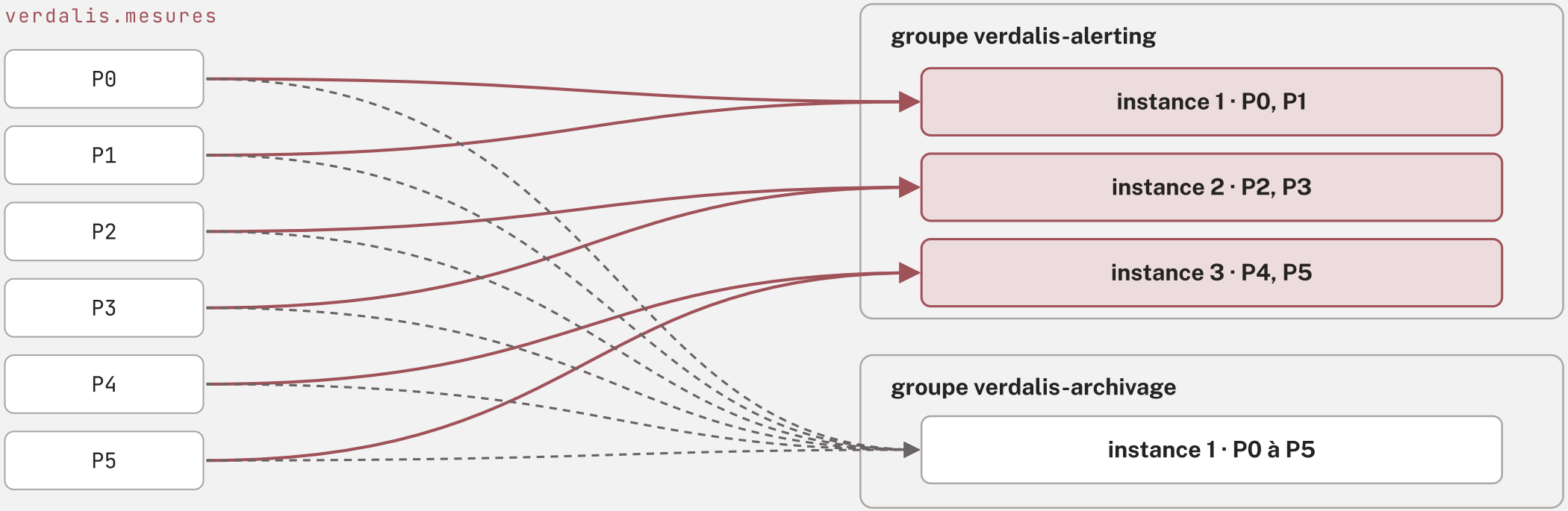
Le pull évite de noyer un consommateur lent : c'est lui qui règle la cadence.

```
while True:
    message = poll()        # « qu'y a-t-il de nouveau ? »
    traiter(message)
    commit(message)         # « j'ai fini jusque-là »
```

DANS LA VRAIE VIE

Le **marque-page**. Chaque lecteur a le sien, et le livre ne change pas quand on le lit.

Le groupe : se partager le travail, sans se gêner entre équipes



Dans un groupe, chaque partition est lue par **un seul** membre.
Entre groupes, chacun lit tout, avec ses propres offsets.

ATTENTION

6 partitions ⇒ 6 consommateurs actifs au maximum dans un groupe. Le 7e reste inactif. Le nombre de partitions plafonne le parallélisme.

CONCEPT REBALANCE

Quand un membre arrive ou part, on redistribue

- Un **coordinateur** (un broker) suit les membres de chaque groupe grâce à des battements de cœur.
- Un membre rejoint, quitte ou ne répond plus : le coordinateur **réattribue** les partitions.
- Pendant l'ancien protocole, tout le groupe s'arrêtait brièvement (« stop the world »).
- Kafka 4.0 généralise un nouveau protocole (KIP-848) : seules les partitions concernées bougent.

DANS LA VRAIE VIE

Un **restaurant** : chaque serveur a ses tables. Un serveur part en pause, le maître d'hôtel répartit ses tables entre les autres. Un nouveau arrive, il récupère quelques tables.

At-most-once, at-least-once, exactly-once

GARANTIE	ORDRE DES OPÉRATIONS	EN CAS DE CRASH	USAGE
Au plus une fois	commit, *puis* traitement	Message perdu	Métriques non critiques
Au moins une fois	traitement, *puis* commit	Message traité deux fois	Le cas par défaut
Exactement une fois	transactions Kafka	Ni perte ni doublon	Kafka → Kafka (Streams)

EN PRATIQUE

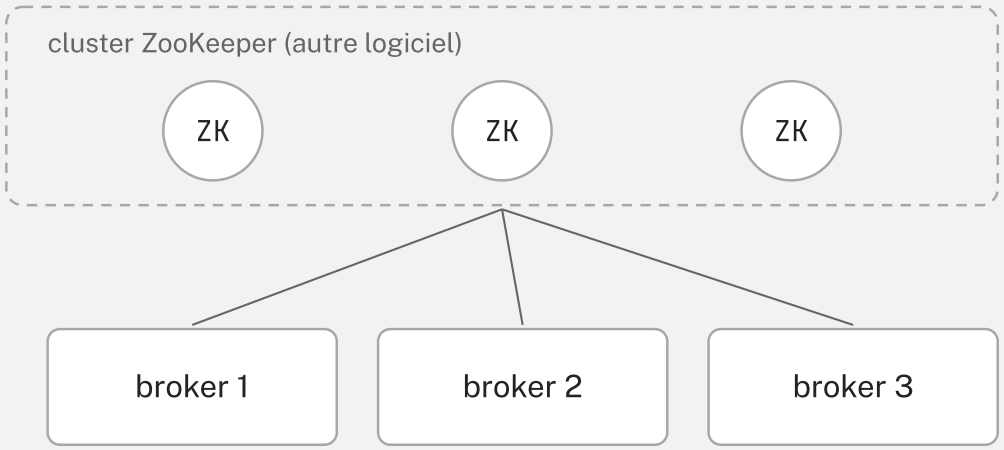
On vise « au moins une fois », et on rend le traitement **idempotent** : traiter deux fois la même mesure ne doit pas créer deux alertes différentes, ou alors ce n'est pas grave.

DANS LA VRAIE VIE

Payer au restaurant : mieux vaut que le serveur vérifie deux fois votre paiement (doublon détecté) que de l'oublier.

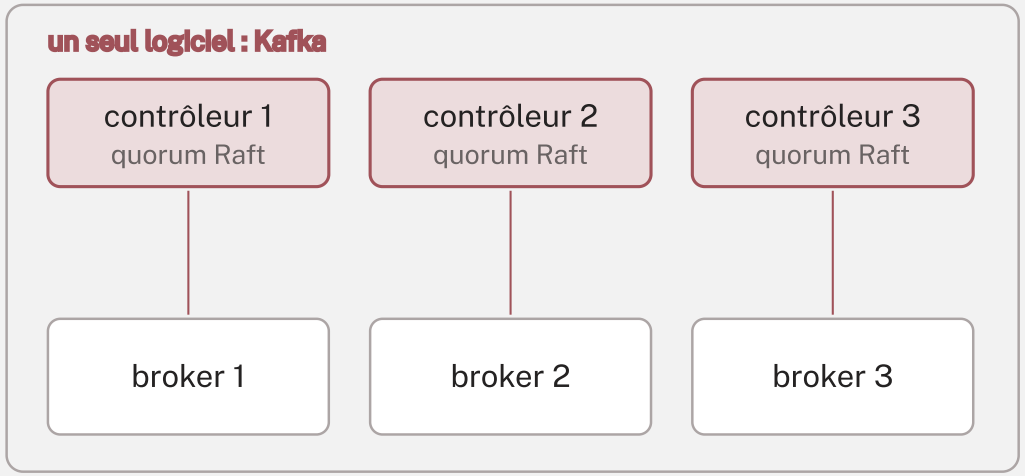
Qui coordonne le cluster ? Hier ZooKeeper, aujourd'hui KRaft

HIER · Kafka + ZooKeeper



2 systèmes à installer, sécuriser, surveiller

AUJOURD'HUI · KRaft (Kafka 4.x)



métadonnées stockées dans un log Kafka interne

Les **contrôleurs** gèrent les métadonnées : quels topics existent, qui est leader de quelle partition, quels brokers sont vivants. Ils s'accordent par vote (protocole **Raft**).

MAJORITÉ

3 contrôleurs tolèrent 1 panne, 5 en tolèrent 2. Il faut toujours une majorité : `(n / 2) + 1`.

ÉCOSYSTÈME AUTOUR DE KAFKA

Ce qui gravite autour du cœur

APACHE

Kafka Connect

Brancher bases, fichiers, S3, Elasticsearch sans écrire de code, via des connecteurs.

APACHE

Kafka Streams

Bibliothèque Java pour filtrer, agréger et joindre des flux.

CLIENTS

Java, librdkafka

Le client Java est officiel. librdkafka (C) sert de base aux clients Python, Go, .NET.

CONFLUENT · AUTRES

Schema Registry

Gérer et faire évoluer les formats de messages (Avro, Protobuf, JSON Schema).

CONFLUENT

ksqlDB · Flink

Traiter des flux en SQL. Apache Flink est très utilisé avec Kafka.

OPEN SOURCE

Kafbat UI

Interface web pour explorer un cluster. On l'installe cet après-midi.

La plateforme Verdalis, en une slide

NOTION	CHOIX POUR VERDALIS	POURQUOI
Cluster	3 nœuds broker + contrôleur	Tolère la panne d'un nœud
Topic	`verdalis.mesures`, `verdalis.alertes`	Un topic par type d'événement
Partitions	6 pour les mesures	Jusqu'à 6 instances de traitement en parallèle
Clé	`serre_id`	L'ordre compte par serre
Réplication	RF 3, min ISR 2	Aucune perte si un broker tombe
Producteur	`acks=all`, idempotent	Ni perte ni doublon à l'envoi
Groupes	`alerting`, `archivage`, `dashboard`	Chaque usage lit tout, à son rythme

Le glossaire de poche

Broker	Serveur Kafka qui stocke et sert les données
Cluster	Ensemble de brokers qui coopèrent
Topic	Catégorie nommée d'événements
Partition	Morceau ordonné d'un topic, unité de parallélisme
Offset	Position d'un événement dans sa partition
Clé	Décide de la partition, donc de l'ordre
Rétention	Durée de conservation des événements

Leader	Copie qui reçoit lectures et écritures
Follower	Copie qui réplique le leader
ISR	Copies à jour, éligibles au leadership
acks	Niveau de confirmation attendu par le producteur
Groupe	Consommateurs qui se partagent les partitions
Commit	Enregistrer l'offset atteint par un groupe
KRaft	Coordination par contrôleurs, sans ZooKeeper

QUIZ CLIQUEZ SUR UNE QUESTION POUR VOIR LA RÉPONSE

Six questions avant le déjeuner

Q1 Un consommateur lit un message. Le message est-il supprimé du topic ?

Q2 Le topic a 4 partitions. On lance 6 consommateurs dans le même groupe. Combien travaillent ?

Q3 Comment garantir que les mesures d'une même serre sont lues dans l'ordre ?

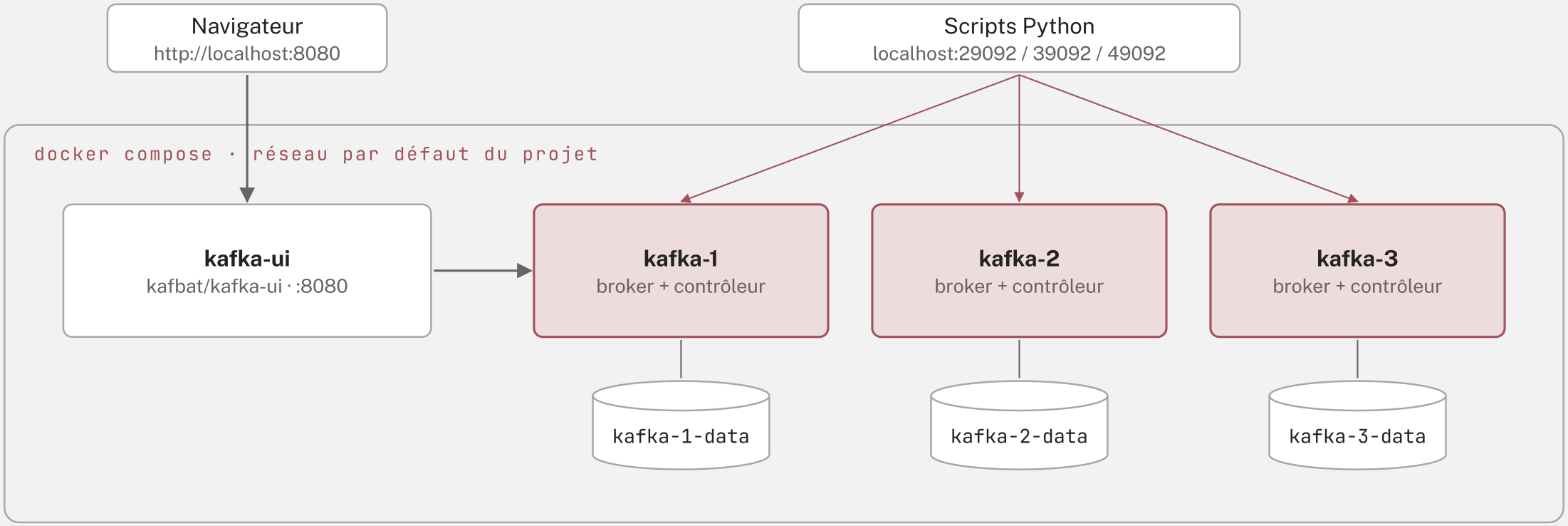
Q4 RF = 3 et min.insync.replicas = 2. Deux brokers tombent. Un producteur en acks=all peut-il écrire ?

Q5 Un tutoriel démarre un conteneur ZooKeeper. Qu'en déduisez-vous ?

Q6 On commite avant de traiter, puis le programme plante. Quelle garantie a-t-on ?

TRANSITION CET APRÈS-MIDI

Assez de théorie. Cet après-midi, on construit le cluster de Verdalis.



PARTITION 1 JOUR 1 · APRÈS-MIDI

Construire

Un cluster Kafka 4 de trois nœuds, une interface web, et nos premiers messages. Chaque ligne de configuration est expliquée.

P0 P1 P2

On monte le cluster par couches

1. **Un seul nœud** (14 h 00). Le minimum qui démarre, pour comprendre chaque variable.
2. **Trois nœuds** (15 h 00). La vraie tolérance aux pannes, et une configuration factorisée.
3. **Kafbat UI** (15 h 30). Voir les brokers, topics, messages et groupes.
4. **Premiers messages** (16 h 00). En ligne de commande, puis en Python, puis on casse des choses.

MÉTHODE

Le formateur tape, vous tapez en même temps. Un fichier qui ne démarre pas chez vous = on s'arrête et on regarde ensemble. Les erreurs de configuration sont le meilleur moment pour apprendre.

Ce dont on a besoin dans Compose

SERVICES

Un conteneur par nœud

`kafka-1`, `kafka-2`, `kafka-3`, `kafka-ui`.

RÉSEAU

DNS par nom de service

Compose crée un réseau pour le projet. Dans ce réseau, `kafka-2` est un nom d'hôte valide. Depuis votre machine, il ne l'est pas.

VOLUMES

Des données qui survivent

Un volume nommé garde les données après `docker compose down`. `down -v` les supprime.

HEALTHCHECK

Attendre que ce soit prêt

`depends_on` + `condition: service_healthy` : l'UI démarre quand Kafka répond vraiment.

ATTENTION

Le point réseau est la source n° 1 des problèmes Kafka sous Docker. Retenez-le : on en reparle dans 10 minutes avec les listeners.

Quelle image Docker pour Kafka ?

IMAGE	ÉDITEUR	NOTRE AVIS
`apache/kafka:4.3.1`	Projet Apache (officielle)	Notre choix. Kafka pur, configuration par variables `KAFKA_*`.
`apache/kafka-native`	Projet Apache	Compilée avec GraalVM : démarre en une seconde. Réservée aux tests, pas à la production.
`confluentinc/cp-kafka`	Confluent	Très répandue dans les tutoriels. Conventions de variables proches mais pas identiques.
`kafbat/kafka-ui:v1.5.0`	Communauté Kafbat	L'interface web. Suite active du projet `provectus/kafka-ui`, qui n'est plus maintenu.

TOUJOURS FIXER LA VERSION

Jamais `:latest` en formation ni en production. Une nouvelle version publiée pendant la nuit ne doit pas changer votre cluster au matin.

Le plus petit cluster qui fonctionne

compose-1-noeud.yml

COPIER

```yaml
services:
  kafka:
    image: apache/kafka:4.3.1
    container_name: kafka
    hostname: kafka
    ports:
      - "29092:9092"          # hôte:conteneur (listener EXTERNAL)
    environment:
      # --- Identité ---
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      CLUSTER_ID: "4L6g3nShT-eMCtK--X86sw"
      # --- Réseau ---
      KAFKA_LISTENERS: INTERNAL://:19092,EXTERNAL://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka:19092,EXTERNAL://localhost:29092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:P
      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      # --- Topics internes : 1 seul nœud ⇒ 1 seule copie ---
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
```

Un seul conteneur, qui joue à la fois **broker** et **contrôleur**.

Les variables sont groupées en quatre familles. On les prend une par une dans les slides suivantes :

1. Identité
2. Réseau (listeners)
3. Topics internes
4. Stockage

Chaque variable `KAFKA_*` devient une propriété Kafka

DANS LA PROPRIÉTÉ	DANS LA VARIABLE
`.` (point)	`_`
`_` (souligné)	`__`
`-` (tiret)	`___`

```
KAFKA_NODE_ID              →  node.id
KAFKA_LOG_DIRS             →  log.dirs
KAFKA_MIN_INSYNC_REPLICAS  →  min.insync.replicas
CLUSTER_ID                 →  (pas de préfixe, cas à part)
```

ATTENTION

Dès que vous passez une variable `KAFKA_*`, l'image n'utilise plus son fichier `server.properties` par défaut. Ce sont alors les valeurs par défaut du code Kafka qui s'appliquent. Il faut donc tout déclarer, y compris les facteurs de réplication des topics internes.

Sans aucune variable, l'image démarre un nœud de démonstration préconfiguré, accessible uniquement sur `localhost:9092` à l'intérieur du conteneur.

ÉTAPE 1 · FAMILLE 1 IDENTITÉ

Qui suis-je, quel rôle, dans quel cluster ?

VARIABLE	VALEUR	RÔLE
KAFKA_NODE_ID	`1`	Identifiant unique du nœud dans le cluster. Deux nœuds avec le même id : le cluster refuse.
KAFKA_PROCESS_ROLES	`broker,controller`	`broker` stocke les données, `controller` vote et gère les métadonnées. Les deux : mode combiné, idéal en développement.
CLUSTER_ID	`4L6g3nSh…`	Identifiant du cluster, écrit sur le disque au premier démarrage. Tous les nœuds doivent avoir le même.

```
# Générer un identifiant de cluster
docker run --rm apache/kafka:4.3.1 \
  /opt/kafka/bin/kafka-storage.sh random-uuid
```

DANS LA VRAIE VIE

Le `CLUSTER_ID`, c'est le **nom de famille** gravé sur la boîte aux lettres. Si un nœud arrive avec un autre nom sur un disque déjà marqué, il refuse de démarrer.

Les listeners : la partie qui fait perdre des heures

Un **listener** est une porte d'entrée du broker : un nom, un port, un protocole.

- `KAFKA_LISTENERS` : les portes **ouvertes**, là où le broker écoute.
- `KAFKA_ADVERTISED_LISTENERS` : les adresses **annoncées** aux clients, pour qu'ils le rappellent.

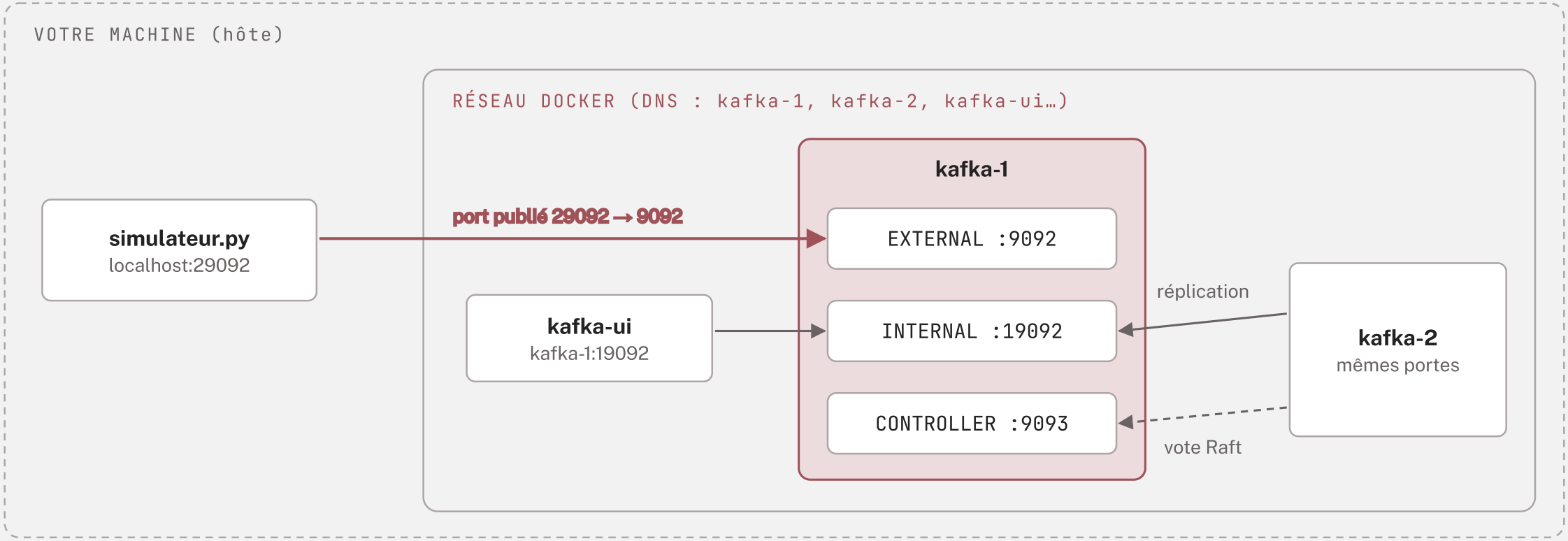
On en déclare trois, car trois publics différents nous parlent :

`INTERNAL`	les autres brokers et Kafbat UI, dans Docker
`EXTERNAL`	vos scripts Python, sur votre machine
`CONTROLLER`	les votes entre contrôleurs

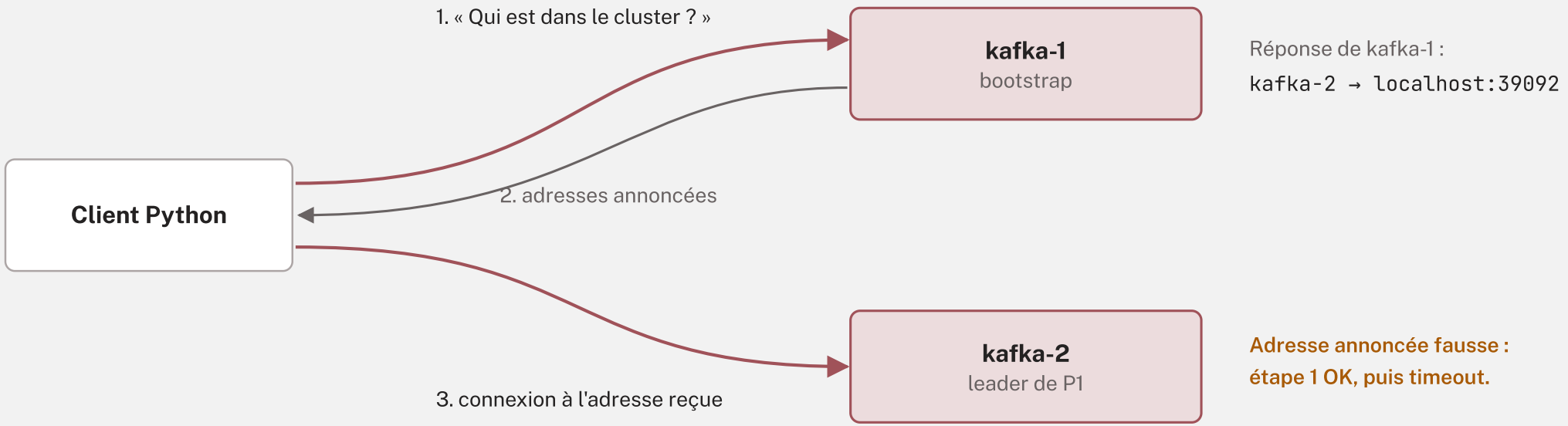
DANS LA VRAIE VIE

Un **immeuble de bureaux**. `LISTENERS`, ce sont les portes : l'entrée principale, l'accès parking, l'entrée de service. `ADVERTISED`, c'est l'adresse imprimée sur la carte de visite. Si la carte donne une adresse que le visiteur ne peut pas atteindre, il reste dehors, même si la porte est grande ouverte.

Deux mondes : votre machine, et le réseau Docker



Le client se connecte deux fois



CONSÉQUENCE

Le client passe par le listener sur lequel il est arrivé et reçoit l'adresse annoncée **pour ce listener**. Arrivé par `EXTERNAL`, il reçoit `localhost:39092`. Arrivé par `INTERNAL`, il reçoit `kafka-2:19092`.

Les variables réseau, ligne par ligne

VARIABLE	VALEUR	RÔLE

`KAFKA_LISTENERS`	`INTERNAL://:19092`
	`EXTERNAL://:9092`
	`CONTROLLER://:9093`
		Ports d'écoute dans le conteneur. Hôte vide = toutes les interfaces.

`KAFKA_ADVERTISED_LISTENERS`	`INTERNAL://kafka:19092` `EXTERNAL://localhost:29092`	Adresses données aux clients. Pour `EXTERNAL` : le port **côté hôte** (29092), pas 9092.
`KAFKA_LISTENER_SECURITY_PROTOCOL_MAP`	`INTERNAL:PLAINTEXT,…`	Protocole de chaque listener. `PLAINTEXT` = ni chiffrement ni authentification.
`KAFKA_INTER_BROKER_LISTENER_NAME`	`INTERNAL`	La porte utilisée entre brokers pour la réplication.
`KAFKA_CONTROLLER_LISTENER_NAMES`	`CONTROLLER`	La porte réservée aux contrôleurs. Jamais annoncée aux clients.
`KAFKA_CONTROLLER_QUORUM_VOTERS`	`1@kafka:9093`	Liste des votants : `id@hôte:port`. Identique sur tous les nœuds.

ATTENTION

`PLAINTEXT` est acceptable en formation. En production : `SASL_SSL` (chiffrement TLS + authentification).

Kafka stocke son propre état… dans des topics

VARIABLE	1 NŒUD	RÔLE
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR	1	Copies de `__consumer_offsets`, où sont stockés les commits des groupes.
KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR	1	Copies de `__transaction_state` (transactions, exactly-once).
KAFKA_TRANSACTION_STATE_LOG_MIN_ISR	1	Répliques à jour exigées pour ce topic.
KAFKA_SHARE_COORDINATOR_STATE_TOPIC_…	1	Même chose pour `__share_group_state` (files d'attente, Kafka 4).
KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS	0	Délai avant le premier rééquilibrage d'un groupe (3 s par défaut). À 0, on voit l'effet tout de suite.

ATTENTION

La valeur par défaut est 3 copies. Avec un seul broker, impossible de créer 3 copies : les consommateurs restent bloqués sans erreur claire. D'où le `1` ici, et le `3` à l'étape 2.

Où vont les données sur le disque ?

```
    environment:
      KAFKA_LOG_DIRS: /var/lib/kafka/data
    volumes:
      - kafka-data:/var/lib/kafka/data

volumes:
  kafka-data:
```

`log.dirs` est le dossier des partitions. On le fait pointer vers un **volume nommé** pour que les données survivent au conteneur.

ATTENTION

Sans ce réglage, Kafka écrit dans `/tmp/kraft-combined-logs`, en dehors du volume prévu par l'image. Supprimer le conteneur efface alors tous vos messages.

POURQUOI /VAR/LIB/KAFKA/DATA

L'image crée ce dossier avec les bons droits pour son utilisateur (`appuser`). Un autre chemin monté en volume peut provoquer un « Permission denied ».

Dans le volume, un dossier par partition : `verdalis.mesures-0`, `verdalis.mesures-1`…

ÉTAPE 1 ON DÉMARRE

Démarrer, lire les logs, interroger le cluster

COPIER

```
docker compose up -d
docker compose logs -f kafka          # Ctrl+C pour sortir

# Le quorum KRaft est-il formé ?
docker exec -it kafka /opt/kafka/bin/kafka-metadata-quorum.sh \
  --bootstrap-server kafka:19092 describe --status

# Quels topics existent ?
docker exec -it kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka:19092 --list
```

Dans les logs, cherchez :

`Kafka Server started`

Dans le statut du quorum :

`LeaderId: 1` et `CurrentVoters` avec votre nœud.

PREMIER RÉFLEXE

Quelque chose ne va pas ? `docker compose logs` avant toute autre chose. Kafka est bavard, et l'erreur est presque toujours écrite en clair.

Pourquoi 3, et pas 2 ?

CONTRÔLEURS	MAJORITÉ	PANNES TOLÉRÉES
1	1	0
2	2	0 (pire que 1 !)
3	2	1
5	3	2

Le quorum Raft a besoin d'une **majorité**. Avec 2 nœuds, la panne de l'un bloque tout. Il faut donc un nombre **impair**.

Côté données, 3 brokers permettent RF = 3 et `min.insync.replicas` = 2.

DANS LA VRAIE VIE

Un **jury de trois** peut toujours trancher à la majorité. Un jury de deux se bloque au premier désaccord.

ÉTAPE 2 NE PAS SE RÉPÉTER

Une configuration commune, écrite une seule fois

docker-compose.yml · lignes 6-22 COPIER

```yaml
x-kafka-env: &kafka-env
  KAFKA_PROCESS_ROLES: broker,controller
  CLUSTER_ID: "4L6g3nShT-eMCtK--X86sw"
  KAFKA_LISTENERS: INTERNAL://:19092,EXTERNAL://:9092,CONTROLLER://:9093
  KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:PLAINTE
  KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
  KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
  KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka-1:9093,2@kafka-2:9093,3@kafka-3:9093
  KAFKA_DEFAULT_REPLICATION_FACTOR: 3
  KAFKA_MIN_INSYNC_REPLICAS: 2
  KAFKA_NUM_PARTITIONS: 3
  KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
  KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
  KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 3
  KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 2
  KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
  KAFKA_LOG_DIRS: /var/lib/kafka/data
```

`x-kafka-env` : un bloc ignoré par Compose (préfixe `x-`), qui sert de modèle.

`&kafka-env` pose une **ancre** sur ce bloc.

`<<: *kafka-env` le **fusionne** dans chaque service. On n'ajoute ensuite que ce qui diffère.

VÉRIFIER LE RÉSULTAT

`docker compose config` affiche le fichier final, ancres résolues.

Chaque nœud : son id, son port, son adresse annoncée

docker-compose.yml · lignes 32-44 COPIER

```yaml
  kafka-1:
    image: apache/kafka:4.3.1
    container_name: kafka-1
    hostname: kafka-1
    ports:
      - "29092:9092"
    environment:
      <<: *kafka-env
      KAFKA_NODE_ID: 1
      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka-1:19092,EXTE
    volumes:
      - kafka-1-data:/var/lib/kafka/data
    healthcheck: *kafka-healthcheck
```

NŒUD	NODE.ID	PORT HÔTE	ADRESSE EXTERNE ANNONCÉE
kafka-1	1	29092	`localhost:29092`
kafka-2	2	39092	`localhost:39092`
kafka-3	3	49092	`localhost:49092`

Dans le conteneur, les trois écoutent sur les mêmes ports (9092, 19092, 9093). Sur votre machine, chacun a **son propre port**, car deux conteneurs ne peuvent pas publier le même.

ATTENTION

Copier-coller un bloc et oublier de changer l'adresse annoncée : le client est renvoyé vers le mauvais broker. Erreur très fréquente.

Les nouvelles valeurs, maintenant qu'on a trois brokers

VARIABLE	VALEUR	RÔLE
KAFKA_CONTROLLER_QUORUM_VOTERS	`1@kafka-1:9093,2@…,3@…`	Les trois votants, sur tous les nœuds.
KAFKA_DEFAULT_REPLICATION_FACTOR	`3`	RF par défaut d'un nouveau topic.
KAFKA_MIN_INSYNC_REPLICAS	`2`	Écriture `acks=all` acceptée si au moins 2 copies sont à jour.
KAFKA_NUM_PARTITIONS	`3`	Partitions par défaut d'un nouveau topic (défaut Kafka : 1).
KAFKA_AUTO_CREATE_TOPICS_ENABLE	`"false"`	Interdit la création automatique d'un topic à la première écriture.
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR	`3`	Les commits des groupes survivent à la perte d'un broker.
KAFKA_TRANSACTION_STATE_LOG_MIN_ISR	`2`	Même logique que `min.insync.replicas`.

POURQUOI INTERDIRE LA CRÉATION AUTOMATIQUE ?

Une faute de frappe (`verdalis.mesure` au lieu de `verdalis.mesures`) créerait silencieusement un nouveau topic. On préfère une erreur claire.

Comment Docker sait-il que Kafka est prêt ?

docker-compose.yml · lignes 24-29 COPIER

```yaml
x-kafka-healthcheck: &kafka-healthcheck
  test: ["CMD-SHELL", "/opt/kafka/bin/kafka-cluster.sh cluster-id --boo
  interval: 10s
  timeout: 10s
  retries: 12
  start_period: 20s
```

- La commande demande l'identifiant du cluster au broker local. S'il répond, il est prêt.
- `start_period` : pendant 20 s, les échecs ne comptent pas (démarrage de la JVM).
- 12 essais × 10 s : au-delà de 2 minutes, le conteneur est déclaré `unhealthy`.

DANS LA VRAIE VIE

Un conteneur « démarré » n'est pas un service « prêt ». C'est la différence entre un restaurant dont la porte est ouverte et un restaurant dont la cuisine est prête à servir.

Visible avec `docker compose ps` : colonne STATUS, `(healthy)`.

ÉTAPE 3 KAFBAT UI

Une interface web pour voir ce qui se passe

docker-compose.yml · lignes 74-89 COPIER

```yaml
  kafka-ui:
    image: kafbat/kafka-ui:v1.5.0
    container_name: kafka-ui
    ports:
      - "8080:8080"
    environment:
      KAFKA_CLUSTERS_0_NAME: verdalis
      KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka-1:19092,kafka-2:19092,ka
      DYNAMIC_CONFIG_ENABLED: "true"
    depends_on:
      kafka-1:
        condition: service_healthy
      kafka-2:
        condition: service_healthy
      kafka-3:
        condition: service_healthy
```

VARIABLE	RÔLE
`KAFKA_CLUSTERS_0_NAME`	Nom affiché. Le `0` est l'index : on peut déclarer plusieurs clusters.
`KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS`	Adresses **internes** (`kafka-1:19092`) : l'UI est dans le réseau Docker.
`DYNAMIC_CONFIG_ENABLED`	Autorise à ajouter ou modifier des clusters depuis l'interface.
`depends_on … healthy`	L'UI attend que les 3 brokers soient prêts.

Le fichier complet, à copier

docker-compose.yml

COPIER

```yaml
# Formation Kafka — cluster Verdalis : 3 nœuds KRaft + Kafbat UI
# Démarrer :  docker compose up -d
# Arrêter  :  docker compose down        (garde les données)
# Tout effacer : docker compose down -v  (supprime les volumes !)

x-kafka-env: &kafka-env
  KAFKA_PROCESS_ROLES: broker,controller
  CLUSTER_ID: "4L6g3nShT-eMCtK--X86sw"
  KAFKA_LISTENERS: INTERNAL://:19092,EXTERNAL://:9092,CONTROLLER://:9093
  KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT
  KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
  KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
  KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka-1:9093,2@kafka-2:9093,3@kafka-3:9093
  KAFKA_DEFAULT_REPLICATION_FACTOR: 3
  KAFKA_MIN_INSYNC_REPLICAS: 2
  KAFKA_NUM_PARTITIONS: 3
  KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
  KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
  KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 3
  KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 2
  KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
  KAFKA_LOG_DIRS: /var/lib/kafka/data

x-kafka-healthcheck: &kafka-healthcheck
```

Démarrer et contrôler les trois nœuds

COPIER

```
docker compose down -v        # repartir d'un disque propre
docker compose up -d
docker compose ps             # attendre 3 × (healthy)

docker exec -it kafka-1 /opt/kafka/bin/kafka-metadata-quorum.sh \
  --bootstrap-server kafka-1:19092 describe --status

# Raccourci pour la suite de l'après-midi
alias kt='docker exec -it kafka-1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server ka
```

Résultat attendu du statut :

- `LeaderId` : 1, 2 ou 3
- `CurrentVoters` : les 3 nœuds
- `MaxFollowerLag` : proche de 0

Puis ouvrez **http://localhost:8080**.

ÉTAPE 3 VISITE GUIDÉE

Ce qu'on regarde dans Kafbat UI

DASHBOARD

État du cluster

En ligne ou non, nombre de brokers, partitions en sous-réplication.

BROKERS

Les 3 nœuds

Lequel est contrôleur actif, espace disque, configuration effective de chaque broker.

TOPICS

Créer et inspecter

Partitions, leaders, ISR, configuration du topic. Créer un topic en 3 clics.

TOPICS › MESSAGES

Lire et produire

Parcourir par partition et offset, filtrer, et produire un message à la main.

CONSUMERS

Groupes et lag

Membres, partitions attribuées, retard par partition. Réinitialiser les offsets.

CONFIG

Réglages dynamiques

Modifier certains réglages de topic ou de broker sans redémarrer.

DÉMO EN DIRECT

Créer `verdalis.test` depuis l'UI (3 partitions, RF 3), y produire un message, le retrouver dans l'onglet Messages avec sa partition et son offset.

Créer `verdalis.mesures` et lire sa description

COPIER

```
kt --create --topic verdalis.mesures --partitions 6 --replication-factor 3
kt --describe --topic verdalis.mesures
```

```
Topic: verdalis.mesures  PartitionCount: 6  ReplicationFactor: 3
  Partition: 0   Leader: 2   Replicas: 2,3,1   Isr: 2,3,1
  Partition: 1   Leader: 3   Replicas: 3,1,2   Isr: 3,1,2
  Partition: 2   Leader: 1   Replicas: 1,2,3   Isr: 1,2,3
  ...                                (sortie simplifiée)
```

`Leader`	Le broker qui sert cette partition	`Replicas`	Où sont les copies ; le premier est le leader « préféré »
`Isr`	Copies à jour. Si Isr = Replicas, tout va bien	Répartition	Les leaders sont répartis sur les 3 brokers

Deux terminaux : l'un écrit, l'autre lit

Terminal A · producteur (clé|valeur)

COPIER

```
docker exec -it kafka-1 /opt/kafka/bin/kafka-console-producer.sh \
  --bootstrap-server kafka-1:19092 --topic verdalis.mesures \
  --reader-property parse.key=true \
  --reader-property 'key.separator=|'

> serre-07|{"type":"temperature","valeur":24.1}
> serre-07|{"type":"temperature","valeur":25.3}
> serre-12|{"type":"humidite","valeur":61}
```

Terminal B · consommateur dans un groupe

COPIER

```
docker exec -it kafka-1 /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server kafka-1:19092 --topic verdalis.mesures \
  --group verdalis-console --from-beginning \
  --formatter-property print.key=true \
  --formatter-property print.partition=true \
  --formatter-property print.offset=true
```

À OBSERVER

Les deux messages de `serre-07` sont dans la même partition, avec des offsets qui se suivent.

Avant Kafka 4.2, ces options s'appelaient `--property`. Vous les verrez partout dans les tutoriels ; elles marchent encore, avec un avertissement.

Où en est chaque groupe ? Le lag

```
docker exec -it kafka-1 /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server kafka-1:19092 --describe --group verdalis-console
```

COPIER

```
GROUP            TOPIC             PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
verdalis-console verdalis.mesures  2          2               2               0
verdalis-console verdalis.mesures  5          1               3               2
```

`CURRENT-OFFSET`	Prochain offset que le groupe lira (dernier commit)
`LOG-END-OFFSET`	Prochain offset qui sera écrit
`LAG`	La différence : messages en attente

DANS LA VRAIE VIE

Le **lag**, c'est le nombre d'épisodes d'une série que vous n'avez pas encore regardés. S'il grandit sans cesse, vous n'arriverez jamais à rattraper.

DÉMO TOLÉRANCE AUX PANNES

Arrêtons un broker, puis deux

COPIER

```
# 1. Un broker tombe
docker stop kafka-2
kt --describe --topic verdalis.mesures
#   Isr: 1,3 ... Leader : jamais 2
kt --describe --under-replicated-partitions

# 2. Il revient
docker start kafka-2
#   après quelques secondes : Isr: 2,3,1

# 3. Deux brokers tombent
docker stop kafka-2 kafka-3
#   essayez de produire avec la console...

docker start kafka-2 kafka-3
```

SITUATION	CE QU'ON OBSERVE
1 broker arrêté	Les leaders migrent. Production et lecture continuent.
Retour	Il rattrape son retard et rejoint l'ISR.
2 brokers arrêtés	Le quorum KRaft est perdu (1 votant sur 3). Les écritures échouent ou expirent.

LE MESSAGE À RETENIR

Kafka préfère **refuser** une écriture plutôt que la confirmer sans garantie.

Un producteur Python en 20 lignes

producer_minimal.py

COPIER

```python
import json
from confluent_kafka import Producer

producer = Producer({
    "bootstrap.servers": "localhost:29092,localhost:39092,localhost:49092",
    "acks": "all",                  # attendre les répliques synchronisées
    "enable.idempotence": True,     # pas de doublon en cas de retry
})

def rapport(err, msg):              # appelé quand le broker a répondu
    if err:
        print("Échec :", err)
    else:
        print(f"OK → partition {msg.partition()}, offset {msg.offset()}")

mesure = {"serre_id": "serre-07", "type": "temperature", "valeur": 24.1}
producer.produce("verdalis.mesures",
                 key="serre-07",
                 value=json.dumps(mesure),
                 callback=rapport)

producer.flush(10)                  # vider le tampon avant de quitter
```

- `bootstrap.servers` : les ports **hôte**, car le script tourne sur votre machine.
- `produce()` est **asynchrone** : il remplit le tampon et rend la main.
- Le `callback` dit si l'écriture a réussi. Il est appelé pendant `poll()` ou `flush()`.
- `flush()` attend que tout soit parti. Sans lui, messages perdus.

Un consommateur Python : la boucle `poll()`

consumer_minimal.py

COPIER

```python
import json
from confluent_kafka import Consumer

consumer = Consumer({
    "bootstrap.servers": "localhost:29092,localhost:39092,localhost:49092",
    "group.id": "verdalis-dashboard",    # nom du groupe = identité du lecteur
    "auto.offset.reset": "earliest",     # sans offset commité : depuis le début
    "enable.auto.commit": False,         # on commite nous-mêmes
})
consumer.subscribe(["verdalis.mesures"])

try:
    while True:
        msg = consumer.poll(1.0)          # attend au plus 1 s
        if msg is None:
            continue
        if msg.error():
            print("Erreur :", msg.error())
            continue
        mesure = json.loads(msg.value())
        print(msg.key().decode(), mesure, f"p{msg.partition()}@{msg.offset()}")
        consumer.commit(message=msg, asynchronous=False)
finally:
    consumer.close()                      # quitte le groupe proprement
```

- `group.id` est obligatoire : c'est l'identité du lecteur.
- `auto.offset.reset` ne sert **que** si le groupe n'a encore rien commité.
- On commite **après** le traitement : au moins une fois.
- `close()` prévient le groupe : le rééquilibrage est immédiat.

Commiter à chaque message est lent. On verra demain comment commiter par lots.

Les réglages client à connaître

PRODUCTEUR	CONSEIL
`bootstrap.servers`	Au moins 2 adresses
`acks`	`all`
`enable.idempotence`	`True` (désactivé par défaut en Python)
`linger.ms`	5 à 50 : regrouper les messages
`compression.type`	`lz4` ou `zstd`
`client.id`	Un nom lisible dans les logs

CONSOMMATEUR	CONSEIL
`group.id`	Un nom par usage : `verdalis-alerting`
`auto.offset.reset`	`earliest` ou `latest`, selon le besoin
`enable.auto.commit`	`False` si la perte coûte cher
`max.poll.interval.ms`	Délai max entre deux `poll()` (5 min)
`session.timeout.ms`	Déclaré mort sans signe de vie (45 s)

ATTENTION

Un traitement qui prend plus de 5 minutes entre deux `poll()` fait exclure le consommateur du groupe, puis déclenche un rééquilibrage en boucle.

Les erreurs que vous allez rencontrer

SYMPTÔME	CAUSE PROBABLE	REMÈDE
Le script se connecte puis expire	Adresse annoncée injoignable	Vérifier `ADVERTISED_LISTENERS` et les ports publiés
`port is already allocated`	Port pris par un autre programme	Changer le port hôte **et** l'adresse annoncée
`Invalid cluster.id` **au démarrage**	`CLUSTER_ID` changé, ancien volume	`docker compose down -v`
`Replication factor … larger than available brokers`	RF 3 sur un cluster d'un nœud	RF ≤ nombre de brokers
`Unknown topic or partition`	Topic absent, création auto interdite	Créer le topic, vérifier son nom
Le consommateur ne reçoit rien	Le groupe a déjà commité ; `auto.offset.reset` ignoré	Nouveau `group.id` ou réinitialiser les offsets
Conteneur `Exited (137)`	Mémoire insuffisante	Plus de RAM pour Docker Desktop

RÉCAP CE SOIR, VOUS AVEZ…

Un cluster qui tient debout, et vous savez pourquoi

- Démarré Kafka 4 en KRaft, d'abord sur un nœud, puis sur trois.
- Compris les trois listeners et la différence entre écoute et adresse annoncée.
- Branché Kafbat UI sur le réseau interne.
- Créé un topic, produit, consommé, mesuré le lag.
- Arrêté des brokers et observé l'ISR.
- Écrit un producteur et un consommateur en Python.

POUR DEMAIN MATIN

Gardez votre dossier et votre cluster. Demain, `docker compose up -d`, et on attaque le cas pratique Verdalis.

Vérifiez ce soir que `producer_minimal.py` fonctionne depuis votre machine.

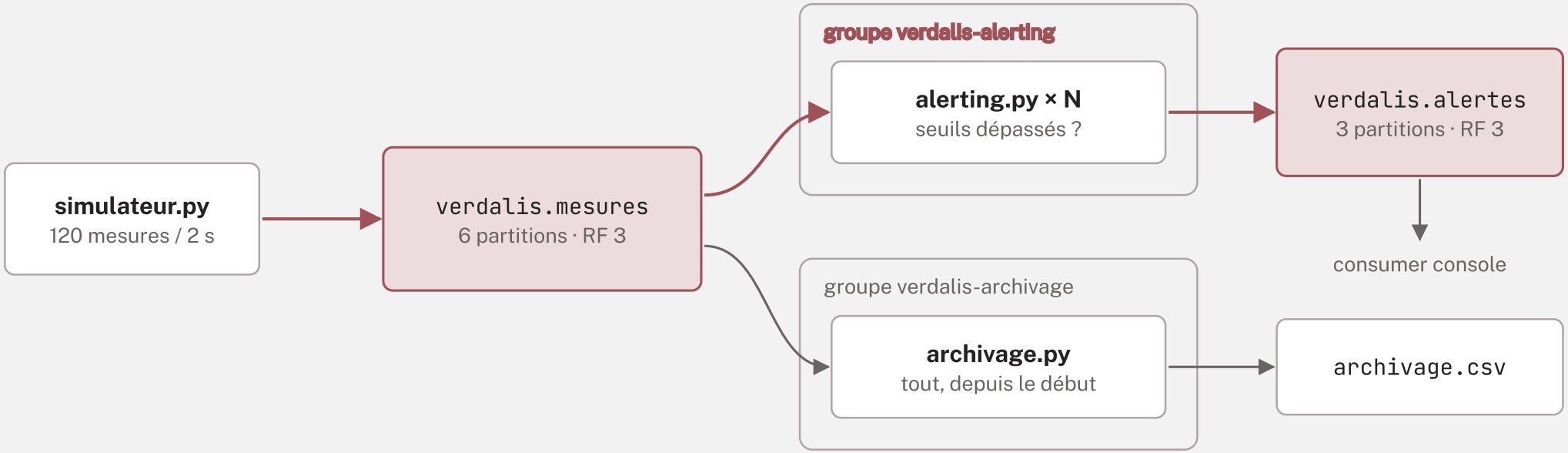
PARTITION 2 JOUR 2 · MATIN

Pratiquer

Verdalis vous confie son premier pipeline : des capteurs jusqu'aux alertes. En binôme, sans le formateur au clavier.

P0 P1 P2

« Prévenez-nous quand une serre surchauffe »



Le technicien de Verdalis veut une alerte dès qu'une serre dépasse un seuil. L'agronome veut, en parallèle, un historique complet dans un fichier. Les deux ne doivent pas se gêner.

Six exercices, une correction commune

HORAIRE	EXERCICE	DURÉE
9 h 00	Présentation, vérification des clusters	10 min
9 h 10	1 · Les topics	15 min
9 h 25	2 · Le simulateur de capteurs	35 min
10 h 00	3 · Le service d'alerte	35 min
10 h 35	Pause	15 min
10 h 50	4 · Passage à l'échelle	20 min
11 h 10	5 · Archivage et rejeu	20 min
—	6 · Bonus : pannes	si le temps le permet
11 h 30	Correction collective	60 min

- Travail en **binôme**, un seul clavier qui tourne à chaque exercice.
- Chaque exercice a des **questions** : notez vos réponses, on les compare à la correction.
- Bloqués plus de 10 minutes ? Levez la main. Chercher fait partie de l'exercice ; rester bloqué, non.
- Les slides d'hier sont votre documentation.

Le contrat : ce que chaque message doit contenir

`verdalis.mesures` · clé = `serre_id`

```
{
  "serre_id": "serre-07",
  "capteur_id": "serre-07-temperature",
  "type": "temperature",
  "valeur": 33.4,
  "unite": "°C",
  "ts": "2026-10-06T09:41:12.503+00:00"
}
```

`verdalis.alertes` · clé = `serre_id`

```
{
  "serre_id": "serre-07", "type": "temperature",
  "valeur": 33.4, "seuil": 32.0,
  "ts_mesure": "2026-10-06T09:41:12.503+00:00",
  "origine": "p2@18234"
}
```

TYPE	VALEUR TYPIQUE	ALERTE SI
`temperature`	24 °C ± 4	> 32 °C
`humidite`	65 % ± 10	< 40 %
`co2`	800 ppm ± 200	> 1 200 ppm

POURQUOI UN CONTRAT ?

Le producteur et le consommateur ne se parlent jamais : le format est leur seul accord. Changer un nom de champ sans prévenir casse tous les lecteurs.

`origine` = partition et offset de la mesure qui a déclenché l'alerte. Pratique pour enquêter.

EXERCICE 1 LES TOPICS

Préparer les deux topics

15 min CLI ou Kafbat UI

À faire

1. Vérifier que `verdalis.mesures` existe avec 6 partitions et RF 3. Sinon, le créer.
2. Créer `verdalis.alertes` : 3 partitions, RF 3, rétention de **30 jours**.
3. Vérifier les deux topics avec `--describe` et dans Kafbat UI.

Questions

- Q1.1 Quelle valeur de `retention.ms` pour 30 jours ?
- Q1.2 Pourquoi garder les alertes plus longtemps que les mesures ?
- Q1.3 Pourquoi moins de partitions pour les alertes ?
- Q1.4 Que se passe-t-il si on crée `verdalis.alertes` avec RF 4 ?

DANS LA VRAIE VIE

Indice : `--config cle=valeur` sur la commande de création.

EXERCICE 2 PRODUCTEUR

Écrire `simulateur.py`

35 min | Python · Producer

Cahier des charges

- 40 serres (`serre-01` à `serre-40`) × 3 capteurs.
- Toutes les 2 secondes, une mesure par capteur (120 messages).
- Valeurs tirées avec `random.gauss(moyenne, écart)`, selon le contrat.
- Clé = `serre_id`, valeur = JSON du contrat.
- `acks=all`, idempotence activée, `linger.ms` et compression.
- Un callback qui compte les échecs.
- `Ctrl+C` arrête proprement, sans perdre de message.

Vérifications

- Dans Kafbat UI, les messages arrivent sur les 6 partitions.
- Toutes les mesures de `serre-07` sont dans la même partition.

Questions

- Q2.1 Pourquoi appeler `poll(0)` régulièrement ?
- Q2.2 Que se passe-t-il si on oublie `flush()` à l'arrêt ?
- Q2.3 Combien de mesures par jour produit ce simulateur ?

EXERCICE 3 CONSOMMER ET PRODUIRE

Écrire `alerting.py`

35 min | Python · Consumer + Producer

Cahier des charges

- Groupe `verdalis-alerting`, abonné à `verdalis.mesures`.
- Pour chaque mesure hors seuil, publier une alerte conforme au contrat dans `verdalis.alertes`.
- Afficher les partitions attribuées au démarrage (`on_assign`).
- Commit **manuel**, uniquement quand les alertes correspondantes sont bien écrites.
- Un message illisible ne doit pas arrêter le service.

Vérification

Un consommateur console sur `verdalis.alertes` affiche les alertes en direct.

Questions

- Q3.1 Le service plante entre l'envoi d'une alerte et le commit. Que se passe-t-il au redémarrage ?
- Q3.2 Et si on commitait *avant* d'envoyer l'alerte ?
- Q3.3 Pourquoi ne pas commiter après chaque message ?

EXERCICE 4 GROUPES ET RÉÉQUILIBRAGE

Et si une seule instance ne suffit plus ?

20 min | Plusieurs terminaux | Kafbat UI › Consumers

Expérience

1. Lancer 1 instance de `alerting.py`. Noter ses partitions.
2. En lancer 2 autres (3 au total). Noter les partitions de chacune.
3. Arrêter une instance avec `Ctrl+C`. Observer.
4. Monter à 7 instances. Observer.

Questions

- Q4.1 Combien de partitions par instance avec 3 instances ?
- Q4.2 Que devient la 7e instance ?
- Q4.3 Les alertes de `serre-07` sont-elles toujours traitées dans l'ordre ? Pourquoi ?
- Q4.4 Verdalis passe à 400 serres. Que faudrait-il changer, et quand aurait-il fallu y penser ?

EXERCICE 5 UN DEUXIÈME GROUPE

Écrire `archivage.py` et rejouer l'historique

20 min | Python · Consumer · CLI

À faire

1. Un consommateur du groupe `verdalis-archivage` écrit chaque mesure dans un CSV : ts, serre, type, valeur, partition, offset.
2. Le lancer pendant que `alerting.py` tourne. Comparer les lags des deux groupes.
3. L'arrêter, le relancer. Relit-il tout ?
4. Faire relire **tout l'historique** au groupe, sans changer son nom.

Questions

- Q5.1 L'archivage ralentit-il les alertes ? Pourquoi ?
- Q5.2 Pourquoi `auto.offset.reset=earliest` ne suffit-il pas pour tout relire la deuxième fois ?
- Q5.3 Jusqu'où peut-on remonter dans le passé ?

DANS LA VRAIE VIE

Indice : `kafka-consumer-groups.sh --reset-offsets` , et le groupe doit être arrêté.

Casser le cluster pendant que tout tourne

Bonus `docker stop / start`

Expérience

1. Simulateur et alerting en marche : `docker stop kafka-2`.
2. Puis `docker stop kafka-3`. Regarder la sortie du simulateur.
3. Redémarrer les deux. Attendre le retour à la normale.
4. Arrêter le simulateur. Comparer son total « envoyées » à la somme des offsets de fin de `verdalis.mesures` (`kafka-get-offsets.sh`).

Questions

- Q6.1 Que remarquez-vous avec un seul broker arrêté ?
- Q6.2 Avec deux ? Les messages sont-ils perdus, ou en attente ?
- Q6.3 Les deux totaux sont-ils égaux ? Qu'est-ce que ça prouve ?
- Q6.4 Qu'aurait changé `acks=1` sans idempotence ?

CAS PRATIQUE AVANT LA CORRECTION

Votre grille d'auto-évaluation

CRITÈRE	VÉRIFICATION
Topics conformes	`--describe` : partitions, RF, rétention
Clés correctes	Une serre = une partition
Aucune perte à l'arrêt	`flush()` à la sortie du simulateur
Alertes justes	Aucune alerte sous le seuil
Commit au bon endroit	Après l'écriture de l'alerte
Robustesse	Un JSON invalide n'arrête pas le service
Groupes indépendants	Deux lags différents dans Kafbat UI

TEST DU MESSAGE PIÈGE

Envoyez à la main, depuis Kafbat UI, un message `serre-99|pas du json` dans `verdalis.mesures`. Votre alerting survit-il ?

À 11 h 30, on corrige ensemble, exercice par exercice. Préparez vos réponses aux questions.

PARTITION 2 · 11 H 30

Correction

Chaque binôme présente une réponse, puis on compare avec le corrigé. Il existe plusieurs bonnes solutions : on discute des compromis.

CONCLUSION BILAN

En une journée et demie, vous avez…

PARTITION 0

Compris

Le log distribué, les topics, partitions et offsets, la réplication, les groupes, les garanties de livraison, KRaft.

PARTITION 1

Construit

Un cluster Kafka 4 de 3 nœuds et Kafbat UI, avec chaque variable de configuration expliquée.

PARTITION 2

Programmé

Un pipeline Verdalis complet : simulateur, alerting, archivage, montée en charge et pannes.

RETOUR AUX ATTENTES DU PREMIER MATIN

Reprenons la liste notée au tableau hier : qu'est-ce qui est couvert, qu'est-ce qui reste ?

Ce qui sépare notre cluster d'un cluster de production

SUJET	EN FORMATION	EN PRODUCTION
Sécurité	`PLAINTEXT`	TLS + SASL, ACL par topic
Rôles	Mode combiné	Contrôleurs dédiés
Machines	3 conteneurs sur un PC	Machines ou zones distinctes
Supervision	Kafbat UI	Métriques JMX, Prometheus, alertes sur le lag
Formats	JSON libre	Schema Registry (Avro, Protobuf)
Hébergement	Docker Compose	Kubernetes (Strimzi) ou service managé

Prochaines notions à explorer

- Kafka Connect, pour brancher une base sans code
- Kafka Streams ou Flink, pour agréger (moyenne par serre et par heure)
- Transactions et exactly-once
- Share groups : Kafka comme file d'attente
- Dead letter queue et gestion des erreurs

Pour continuer seul

RÉFÉRENCE

Documentation Apache Kafka

kafka.apache.org/documentation : configuration, protocole, opérations.

DOCKER

Exemples Docker officiels

github.com/apache/kafka › docker/examples : nœud seul, clusters, SSL, SASL.

PYTHON

confluent-kafka-python

github.com/confluentinc/confluent-kafka-python et la liste des réglages de librdkafka (CONFIGURATION.md).

INTERFACE

Kafbat UI

github.com/kafbat/kafka-ui : configuration, authentification, plusieurs clusters.

Livre de référence : *Kafka: The Definitive Guide* (O'Reilly, 2e édition). Écrit avant Kafka 4 : les chapitres ZooKeeper sont à lire comme de l'histoire.

Merci.
Des questions ?

Le log continue : ajoutez vos propres événements.

offset 18232	offset 18233	offset 18234	offset 18235	offset 18236	offset 18237	offset 18238
serre-24	serre-37	serre-31	serre-24	serre-13	serre-19	serre-28
co2 773 ppm	temperature 27,0 °C	co2 903 ppm	co2 771 ppm	temperature 21,9 °C	temperature 27,8 °C	temperature 24,4 °C